

Bas — TD 2

Commandes shell (suite) · Travaux dirigés · UE Bases de l'Architecture et des Systèmes · L1 Informatique, UPS · Semestre 1 2026-2027

Ce document reprend l'énoncé du TD2 (équipe pédagogique Système, UPS). **Partie 1** : les 5 énoncés seuls, pour s'entraîner sans être influencé par la réponse. **Partie 2** (après le saut de page) : le corrigé rédigé et vérifié de chaque exercice. Commandes de référence : `grep`, `mail`, `cut`, `sort`, `tail`, `find` (cf. TD1 pour `ls`, `echo`, `cat`, `wc`).

Partie 1 — Énoncés

1. Redirections

Exercice 1

Que réalisent les deux commandes suivantes ?

```
echo "bonjour" > cat
echo "bonjour" | cat
```

Exercice 2

Commencer par lire la description de la commande `mail` (cf. document commandes).

- 1) En une seule ligne de commande, envoyer à l'utilisateur `l2inf250`, par courrier électronique, la liste détaillée des répertoires et fichiers contenus dans le répertoire courant.
- 2) Soit un fichier `liste.txt`, contenant une liste d'adresses électroniques enregistrées ligne par ligne, et soit un fichier `lettre.txt` contenant un message à envoyer. Écrire la ligne de commande permettant d'envoyer ce message aux adresses de la liste.

2. Expressions régulières

Exercice 3

Soit le fichier texte non vide `texte.txt`. Utiliser uniquement la commande `grep` pour afficher :

- 1) toutes les lignes,
- 2) les lignes vides,
- 3) les lignes qui commencent par `debut` ou `Debut`,
- 4) les lignes contenant un caractère `$` suivi de deux caractères quelconques suivis d'un caractère `\`,
- 5) les lignes finissant par une lettre minuscule,
- 6) les lignes commençant par dix lettres successives (la première doit être une majuscule et les neuf suivantes doivent être des minuscules) suivies d'un chiffre.

Exercice 4

Sans utiliser les métacaractères du shell à l'exception du métacaractère `|`, écrire la ligne de commande permettant d'afficher :

- 1) les noms de fichiers non cachés du répertoire courant dont le nom contient le caractère `a`,
- 2) les noms de fichiers non cachés du répertoire courant dont le nom comporte quatre caractères exactement,
- 3) les noms de fichiers non cachés du répertoire courant dont le nom comporte au moins cinq caractères,
- 4) les noms de fichiers non cachés du répertoire courant dont le nom commence par une majuscule.

3. Branchement de commandes

Commandes utiles : `tr`, `cut`, `sort`, `tail` (cf. document commandes).

Exercice 5

Réaliser chacune des tâches suivantes à l'aide d'une seule ligne de commande.

- 1) Écrire dans le fichier `exe.txt` les noms des fichiers ordinaires non cachés du répertoire courant ayant l'extension `.sh` et dont le propriétaire n'a pas le droit d'exécution. Si le fichier `exe.txt` existe déjà, son contenu doit être écrasé.
- 2) Afficher le nom du plus gros (celui qui a la plus grande taille) fichier ordinaire non caché du répertoire courant.
- 3) Afficher le nombre d'éléments du répertoire courant (fichiers ordinaires, sous-répertoires ou autres), cachés ou non, dont le nom contient au moins un caractère point.

Partie 2 — Corrigé

1. Redirections

Exercice 1

Commande	Effet
<code>echo "bonjour" > cat</code>	Le <code>></code> est une redirection : sa cible est toujours interprétée comme un nom de fichier , jamais comme une commande à exécuter. Cette commande crée (ou écrase) un fichier ordinaire nommé cat dans le répertoire courant, contenant le texte <code>bonjour</code> . La commande <code>cat</code> (le programme) n'est pas exécutée .
<code>echo "bonjour" cat</code>	Le <code> </code> est un pipe : il connecte la sortie standard de <code>echo</code> à l'entrée standard de la commande <code>cat</code> (le programme, cette fois bien exécuté). <code>cat</code> recopie ce qu'il reçoit sur <code>stdin</code> vers <code>stdout</code> : le texte <code>bonjour</code> s'affiche donc simplement à l'écran. Aucun fichier n'est créé.

Vérifié : après `echo "bonjour" > cat`, un fichier `cat` apparaît bien dans `ls -l` et contient `bonjour` ; la commande `cat` du système reste utilisable normalement juste après (elle n'a pas été remplacée). C'est uniquement si l'on exécutait ensuite `./cat` avec `.` dans le `PATH` que ce fichier local pourrait être lancé par erreur.

Exercice 2

1) Envoyer par email à `l2inf250` la liste détaillée du répertoire courant : `mail` lit le corps du message sur son entrée standard, donc on lui redirige (via un pipe) la sortie de `ls -l` :

```
ls -l | mail l2inf250
```

2) Envoyer le contenu de `lettre.txt` à toutes les adresses listées (une par ligne) dans `liste.txt` :

```
mail `cat liste.txt` < lettre.txt # ou : mail $(cat liste.txt) < lettre.txt
```

Même mécanisme de capture qu'au TD1 (ex. 10) : ``cat liste.txt`` transforme le contenu du fichier (une adresse par ligne) en une liste de **paramètres** — les destinataires de `mail` — tandis que `< lettre.txt` redirige le **corps du message** depuis le fichier `lettre.txt`.

2. Expressions régulières

Exercice 3

Demande	Commande
Toutes les lignes	<code>grep '' texte.txt</code>
Les lignes vides	<code>grep '^\$' texte.txt</code>
Lignes commençant par debut ou Debut	<code>grep '^[Dd]ebut' texte.txt</code>
Lignes avec \$ + 2 car. quelconques + \	<code>grep '\\$.\\.\\' texte.txt</code>
Lignes finissant par une minuscule	<code>grep '[a-z]\$' texte.txt</code>
Maj. + 9 minuscules + un chiffre, en début de ligne	<code>grep '^[A-Z][a-z]{9}[0-9]' texte.txt</code>

- **1)** Le motif vide `' '` est présent (vide) dans toute chaîne : `grep` affiche donc chaque ligne du fichier, sans filtrage.
- **4)** Dans une expression régulière de base (BRE), `$` n'est une ancre de fin de ligne que s'il est le **dernier** caractère du motif ; ici il est suivi d'autre chose, il est donc déjà littéral. Il est tout de même échappé (`\$`) par clarté. Le `\` final doit lui aussi être échappé (`\\`) pour désigner un unique backslash littéral, car `\` est le caractère d'échappement de `grep`.
- **6)** `\{9\}` est la syntaxe BRE (avec `grep` classique, sans `-E`) pour répéter le motif précédent exactement 9 fois.

Toutes ces commandes ont été testées sur un fichier `texte.txt` construit avec des cas couvrant chaque situation (lignes `debut/Debut`, ligne `$xy\`, ligne `Abcdefghij5`, lignes vides, etc.) et donnent exactement le résultat attendu.

Exercice 4

Demande	Commande
Nom contenant le caractère a	<code>ls grep a</code>
Nom de exactement 4 caractères	<code>ls grep '^.\{4\}\$'</code>
Nom d'au moins 5 caractères	<code>ls grep '^.\{5\}+'</code>
Nom commençant par une majuscule	<code>ls grep '^[A-Z]'</code>

Ici c'est `ls` (sans motif de sélection) qui liste les fichiers, et `grep` — seul métacaractère shell autorisé avec le pipe `|` — qui filtre le résultat avec une expression régulière : `^.\{4\}$` ancre le début **et** la fin pour imposer exactement 4 caractères, alors que `^.\{5\}` (sans ancre de fin) impose seulement **au moins** 5 caractères en début de nom. Vérifié sur un jeu de fichiers de test (`abc.txt`, `xyza`, `toto`, `Aaa`, `Bcd`, `four5`, `zzzz12...`).

3. Branchement de commandes

Exercice 5

1) Repérer, dans la sortie de `ls -l`, les fichiers dont le **4e caractère** de la chaîne de droits (le droit d'exécution du propriétaire) est `-`, puis en extraire le nom (9e champ, en supposant les champs de `ls -l` séparés par un seul espace, convention habituelle de ce type d'exercice) :

```
ls -l *.sh | grep '^-...' | cut -d' ' -f9 > exe.txt
```

Le motif `^-...` décompose les 4 premiers caractères de la ligne `ls -l` : `-` (type = fichier ordinaire), 2 caractères quelconques (droits de lecture/écriture du propriétaire), puis `-` (le propriétaire n'a **pas** le droit d'exécution). Le glob `*.sh` exclut déjà naturellement les fichiers cachés (il ne matche pas un `.` initial). `>` écrase `exe.txt` s'il existait déjà (comportement par défaut de `>`).

Vérifié avec 3 fichiers `.sh` de test (un exécutable par le propriétaire, deux non) : seuls les deux non-exécutables par le propriétaire sont retenus. Variante équivalente avec `find` : `find . -maxdepth 1 -type f -name "*.sh" ! -name ".*" ! -perm -u+x` (le `! -name ".*"` est nécessaire ici car, contrairement au glob du shell, `-name` de `find` matche aussi les fichiers cachés par défaut).

2) Ne garder que les fichiers ordinaires (1er caractère `-`), les trier numériquement par taille (5e champ) et prendre le dernier (le plus gros) :

```
ls -l | grep '^-' | sort -k5 -n | tail -1 | cut -d' ' -f9
```

`sort -k5 -n` trie par ordre **croissant** sur le 5e champ (la taille, en tri numérique) ; `tail -1` prend donc la **dernière** ligne, c'est-à-dire le fichier de plus grande taille. Vérifié avec plusieurs fichiers de tailles différentes (0, 5 et 5000 octets) : le fichier de 5000 octets est correctement identifié.

3) Compter les éléments (fichiers, répertoires ou autres, cachés ou non) dont le nom contient un point :

```
ls -a | grep '\.' | wc -l
```

Avec `ls -a`, les entrées spéciales `.` et `..` sont elles aussi listées et contiennent (littéralement) un point : elles sont donc comptées dans ce résultat, conformément à l'énoncé qui ne les exclut pas explicitement. Si l'on veut ne compter que les **vrais** fichiers/répertoires du répertoire (en excluant ces deux entrées spéciales), utiliser `ls -A` à la place de `ls -a` (identique, mais sans `.` ni `..`).

À retenir

- **> vs |** : `>` redirige vers un **fichier** (sa cible n'est jamais interprétée comme une commande, même si elle porte le nom d'une commande existante) ; `|` connecte la sortie d'une commande à l'entrée d'une **autre commande** réellement exécutée.
- **mail** : lit le corps du message sur `stdin` (d'où l'usage d'un pipe ou de `<`) et prend les destinataires en paramètres — la capture ``cat fichier`` permet de transformer un fichier de contenus (adresses, noms...) en liste de paramètres pour une commande.
- **BRE (grep classique)** : `$` n'est une ancre que placé en fin de motif, sinon il est littéral ; les quantificateurs `{n}` et alternatives groupées doivent être échappés (`\{n\}`), contrairement aux expressions étendues (`grep -E`).
- **Filtrer sans métacaractères de glob** : `ls` (sans motif) puis `grep` avec une expression régulière reproduit les mêmes sélections que `*?/[[] — ^.\{n\}$` pour une longueur exacte, `^.\{n\}` (sans ancre finale) pour une longueur minimale.
- **Branchement** `ls -l → grep → sort/cut` : le 1er caractère de `ls -l` donne le type (fichier/répertoire), les 9 caractères suivants les droits (propriétaire/groupe/autres, 3 par 3) ; `sort -k5 -n` trie par taille, `cut -d' ' -fN` extrait un champ — sous réserve que les champs soient séparés par un seul espace, hypothèse habituelle de ce type d'exercice mais pas garantie dans tous les cas réels (noms de groupes longs, tailles à plusieurs chiffres...).